Parallellisering av nätverksanrop i Flutter/Firebase applikationer

Jonathan Karlgren
Elliot Lidman
Teknikprogrammet
Maja Beskowgymnasiet
Umeå, Sverige

Abstract

This study investigates the effectiveness of parallel database calls as a method for improving loading times in a Flutter/Firebase mobile application. Using interleaved testing of both parallel and sequential network calls, the research was conducted on a sample mobile app, focusing on loading times as the key performance metric.

The measurements were performed under controlled conditions, collecting a total of 150 data points for each database size across both the profile and leaderboard pages. To evaluate the method's effectiveness under different load conditions, these tests utilized varying database sizes (100, 500, and 1,000 users).

The results indicate a significant reduction in loading times after implementing parallel database calls. Depending on the page, the improvement ranged from 102 ms to 219 ms; furthermore, depending on both the page and database size, the observed improvement was between 18% and 31%. Statistical analysis shows that the improvement is significant for both pages, with no notable deviations related to database size.

These findings suggest that parallel database calls can be an effective method for improving loading times in Flutter/Firebase mobile applications. However, the results should be interpreted with caution, as the study was conducted on a single application under specific conditions. Further research is needed to generalize these findings to other applications and contexts.

1 Inledning

Laddningstider utgör en central del av användarupplevelsen i moderna mobilapplikationer. Tidigare forskning visar att användare är mycket känsliga för även små fördröjningar; svarstider som överstiger en sekund bryter användarens tankegång, medan fördröjningar på tio sekunder eller mer markant ökar sannolikheten för att applikationen överges. [1] För datadrivna applikationer, där innehåll kontinuerligt hämtas från externa databaser, utgör nätverksanrop en betydande del av den totala laddningstiden.

Trots denna betydelse saknas tydliga riktlinjer för hur databasanrop bör struktureras i Flutter- och Firebase-baserade applikationer för att minimera latens. Detta skapar ett behov av att empiriskt undersöka hur olika implementationsstrategier påverkar laddningstider i praktiken. Inom systemdesign och operativsystem är det väl etablerat att effektiv hantering av I/O-operationer är avgörande för systemets prestanda. [2]

1.1 Produktbeskrivning

Produkten som studeras är mobilapplikationen *Neatify*, utvecklad med ramverket Flutter och med Firebase Firestore som databashanteringssystem. Flutter möjliggör plattformsoberoende utveckling där applikationslogik skrivs i programmeringsspråket Dart. [3]

Neatify syftar till att förbättra användarens produktivitet genom spelifiering, där användaren motiveras att utveckla positiva beteenden genom belöningar och kontinuerlig feedback baserad på interaktioner i applikationen.

Applikationen består av flera vyer som dynamiskt hämtar användardata från Firestore för att rendera innehåll och funktionalitet. Vid navigering mellan dessa vyer utförs flera oberoende nätverksanrop. Särskilt sidorna *Profile* och *Leaderboard* innehåller ett större antal sådana anrop och utgör därför fokus för denna rapport.

1.2 Förbättringsmetod

Den metod som undersöks för att förbättra laddningstiderna i Neatify är parallellisering av nätverksanrop med hjälp av `Future.wait()` i Dart. Istället för att exekvera oberoende anrop sekventiellt, där varje anrop väntar på att det föregående ska slutföras, initieras

samtliga anrop samtidigt och applikationen inväntar deras gemensamma resultat innan rendering sker. [4]

Metoden bygger på principer inom asynkron programmering, där oberoende I/O-operationer kan initieras parallellt för att minska den totala väntetiden. [5] Detta angreppssätt är teoretiskt motiverat då den totala exekveringstiden i nätverksberoende system ofta begränsas av den långsammaste operationen snarare än summan av alla operationer, vilket gör parallellisering till en naturlig strategi för att reducera latens.

1.3 Syfte och frågeställning

Syftet med denna rapport är att empiriskt utvärdera effekten av parallellisering av nätverksanrop i Flutter- och Firebase-baserade mobilapplikationer, med fokus på laddningstider i applikationen Neatify. Utvärderingen avgränsas till sidorna *Profile* och *Leaderboard*, där laddningstid mäts med p95 och medelvärde.

Huvudfrågeställning:

- Är parallellisering av nätverksanrop med `Future.wait()` en lämplig metod för att förbättra laddningstider i en Flutter- och Firebase-baserad applikation som Neatify?

Stödjande frågeställningar:

- Hur stor förbättring av laddningstider (medelvärde och p95) uppnås efter implementering av parallellisering av nätverksanrop på sidorna *Profile* och *Leaderboard*?
- Hur påverkas resultatet av databasstorlek och den nätverkslatens som observerades under mätningarna?
- Är förbättringen statistiskt signifikant och praktiskt relevant?

1.4 Teori

För att förstå parallelliseringens potentiella effekt på laddningstider är det viktigt att först förstå de bakomliggande faktorerna:

1.4.1 Laddningstider i mobilapplikationer

Laddningstid i en mobilapplikation definieras i denna studie som tiden från att en sida initieras till att allt innehåll är renderat och interaktivt. I nätverksberoende applikationer är laddningstiden beroende av den tid det tar att överföra data mellan klient och server.

Denna överföringstid påverkas av nätverkslatens, vilket motsvarar den totala end-to-end-fördröjningen i nätverket. Denna fördröjning består av flera komponenter, inklusive processerings-, kö-, transmissions- och propageringsfördröjningar. [6]

Utöver nätverkslatens påverkas laddningstiden även av hur många nätverksanrop som krävs samt hur dessa organiseras. Varje enskilt anrop medför en ytterligare fördröjning, vilket innebär att flera sekventiella anrop kan leda till att fördröjningarna ackumuleras.

Som en förenklad modell kan den totala laddningstiden därför beskrivas som beroende av nätverkslatens, antal nätverksanrop samt svarstiden för varje enskilt anrop.

När flera nätverksanrop utförs sekventiellt ackumuleras dessa fördröjningar, vilket innebär att den totala laddningstiden approximativt motsvarar summan av varje enskilt anrops svarstid. Genom att istället initiera oberoende anrop parallellt begränsas den totala väntetiden teoretiskt av det långsammaste enskilda anropets svarstid.

1.4.2 Asynkron programering i Dart

Asynkron programmering i Dart görs med klassen `Future` vilket representerar ett potentiellt värde eller error som kommer att vara tillgängligt i framtiden. `Future` kan skapas genom att använda `async` och `await` eller genom att direkt skapa en `Future` med hjälp av dess konstruktor. [5] För att hantera flera oberoende `Future`-objekt kan man använda `Future.wait()`, som tar en lista av `Future`-objekt och returnerar en ny `Future` som fullföljs när alla ingående `Future`-objekt har fullföljts, och som innehåller en lista av resultaten i samma ordning som de ingående `Future`-objekten. [4]

1.4.3 Warm- och cold-boot

Warm-boot och cold-boot är termer som används för att beskriva olika startförhållanden i en applikation. Cold-boot refererar till den initiala starten av en applikation, där alla resurser och data måste laddas från grunden, vilket ofta resulterar i längre laddningstider. Warm-boot är, å andra sidan, när en applikation startas efter en tidigare cold-boot, vilket innebär att vissa resurser och data kan vara lagrade, vilket resulterar i kortare laddningstider.

1.4.4 Flutter och Firebase

Flutter: Flutter är Googles gratis och open-source UI-ramverk för att bygga nativt kompilerade applikationer för mobil, webb och desktop från en enda kodbas. [3]

Firebase: Firebase är en plattform för mobil- och webbapplikationsutveckling som erbjuder en rad tjänster, inklusive realtidsdatabaser, autentisering och hosting. [7] De specifika tjänster som används i Neatify är Firestore, en NoSQL-databas som möjliggör skalbar och flexibel datalagring, och Firebase Authentication för att hantera användarautentisering. [8] [9]

1.4.5 Statistiska mått i hypotesprövningen

I denna studie används ett parat Wilcoxon signed-rank-test för att jämföra sekventiell och parallell implementation. I analyskoden beräknas differensen som $d_i = \text{sekventiell}_i - \text{parallell}_i$ och testet körs med `alternative='greater'`. Noll-differenser exkluderas (`zero_method='wilcox'`).

Den rapporterade *W-statistiken* i denna studie motsvarar summan av ranger för positiva differenser, det vill säga

$$W = W^+ = \sum \text{rang}(|d_i|) \text{ för } d_i > 0.$$

Ett högt W innebär att många och/eller stora positiva differenser förekommer, vilket innebär att observationerna i hög grad ligger i den riktning som definieras av den positiva differensen. Eftersom testet är ensidigt ("greater") prövas nollhypotesen mot en riktad alternativ hypotes i denna riktning.

Som effektstorlek används rangbiseriell korrelation, r_{rb}, som beskriver styrkan och riktningen på skillnaden i den parade jämförelsen. Den kan skrivas som

$$r_{rb} = \frac{W^+ - W^-}{W^+ + W^-},$$

där W^+ är summan av positiva ranger och W^- summan av negativa ranger för de icke-noll differenserna. Måttet varierar mellan -1 och 1, där värden nära 1 indikerar en stark fördel för den parallella implementationen (kortare laddtider), värden nära -1 indikerar motsatsen, och värden nära 0 indikerar liten eller ingen systematisk skillnad.

1.5 Bakgrund

Parallellisering av databasanrop har sin teoretiska grund i forskningsområdet parallel query processing, där målet är att minska exekveringstid genom att utföra flera operationer samtidigt. Tidig forskning inom området visar att databassökningar kan delas upp i deluppgifter som utförs parallellt över flera processorer, vilket möjliggör prestandavinster jämfört med sekventiell exekvering. [10]

I moderna databassystem har dessa principer vidareutvecklats i distribuerade miljöer, där data partitioneras över flera noder och bearbetas parallellt innan resultaten sammanställs. Studier visar att distribuerad parallell bearbetning kan förbättra skalbarhet och svarstider, särskilt när partitioneringen utformas för att minimera kommunikation mellan noder, exempelvis genom grafbaserade metoder. Samtidigt påverkas vinsterna av faktorer som kommunikationskostnader och synkronisering mellan noder. [11]

Forskning inom samtidiga databassystem, inklusive GPU-baserade lösningar, visar att flera förfrågningar kan exekveras parallellt för att bättre utnyttja tillgängliga beräkningsresurser, även om detta kräver effektiv hantering av resurstilldelning och exekveringsplanering. [12] Vidare indikerar studier att parallella exekveringstekniker kan bidra till förbättrad skalbarhet genom att dela upp operationer i samtidiga deluppgifter. [13]

Även om dessa studier främst behandlar CPU-bundna eller distribuerade databassystem, bygger de på grundläggande principer om parallellism, väntetider och resursutnyttjande. Dessa principer är relevanta även i mobilapplikationer, där databasanrop ofta utgörs av nätverksbaserade I/O-operationer. I sådana fall påverkas svarstiden av faktorer som nätverkslatens och serverrespons, och genom att initiera flera oberoende anrop parallellt kan den totala väntetiden potentiellt reduceras. Därmed kan mobilapplikationers databasanrop analyseras utifrån samma övergripande prestandaprinciper som beskrivs i tidigare forskning.

2 Metod

Detta avsnitt beskriver hur mätningarna genomfördes, hur förbättringen implementerades samt hur den statistiska analysen utfördes.

2.1 Implementering av förbättring

Implementeringen av förbättringen bestod av att ersätta de sekventiella Firestore-anropen på *Profile page* och *Leaderboard page* med parallella anrop med hjälp av Darts `Future.wait()`, vars funktion beskrivs närmare i avsnitt 1.4.2.

Förändringen genomfördes uteslutande i de delar av koden som hanterar datahämtning vid sidladdning, utan ändringar i UI-logik, datamodeller eller övrig applikationslogik, för att säkerställa att eventuella skillnader i mätresultaten kan tillskrivas enbart den parallella exekveringen. Implementationen versionshanterades i branch `parallel-db`, isolerad från den ursprungliga koden i `staging`.

Innan de fullskaliga utvärderingsmätningarna genomfördes verifierades implementeringen manuellt genom att navigera till de berörda sidorna och bekräfta att all

data laddades korrekt och att sidorna renderades som förväntat efter övergången till parallell exekvering.

2.2 Genomförande av mätningar

För att kunna besvara frågeställningen togs ett test-script fram, som använder Flutters `IntegrationTestWidgetsFlutterBinding` med en initial cold-boot för varje sida där firestores caching är inaktiverat, för att snabbt och enkelt kunna köra flera mätningar av laddningstider, som mäts från att sidan öppnas till att allt innehåll på sidan är renderat och interaktivt. I kombination med detta test-script kopierades de två relevanta sidorna, *Profile page* och *Leaderboard page*, till samma branch som den parallella implementeringen för att kunna köra ett alternerande mönster av mätningar, där jämna mätningspar kördes med den sekventiella versionen först och udda mätningspar kördes med den parallella versionen först, för att minimera påverkan av nätverksvariation på resultaten då de på så sätt kommer att köras med mer konsistenta förhållanden till varandra utan bias för vilken som körs först. Med test-scriptet gjordes 30 mätningar för varje sida och kördes sedan fem gånger, vilket resulterade i en total mängd av 150 warm-mätningar.

För att säkerställa att mätningarna var så stabila som möjligt, så kördes alla mätningar i en kontrollerad miljö där inga andra program var igång och där telefonen inte användes under mätningarna. Tiderna från mätningarna loggades i en csv-fil för att sedan kunna analyseras med hjälp av Python.

Mätningarna upprepades med olika storlekar på databasen för att kunna utvärdera förbättringens effekt på olika belastningsnivåer. För att kunna kontrollera databasens storlek användes ett seeding-script, skrivet i JavaScript med Node.js, som genererade testdata för databasen i storlekarna 100, 500, 1000 användare. För att säkerställa att databasen var samma, förutom för användarantalet, sattes alla andra `fields` till samma värde för alla mätningar. För varje databasstorlek antecknades både storleken och nätverkshastigheten, som togs fram med `Speedtest av Ookla` med servern satt till Bahnhof AB i Sundsvall (219 km bort), för att kunna kontrollera dessa faktorer i analysen.

För alla mätningar användes en OnePlus 12R med Android 16 och OxygenOS 16.0.3 som testenhet. Testenheten var ansluten till en stabil Wi-Fi-anslutning och hade alla onödiga bakgrundsprocesser stängda för att minimera störningar. För att säkerställa att resultaten var så tillförlitliga som möjligt, så genomfördes alla mätningar under liknande förhållanden, inklusive tid på dagen och batterinivå.

För att säkerställa reproducerbarhet och transparens dokumenterades den exakta kodstatusen som användes under de initiala mätningarna via Git. Branch `parallell-db` innehåller den parallella implementationen tillsammans med kopior av sidorna med sekventiella anrop. Men den innehåller även seeding- och test-scriptet. Dock kräver seeding-scriptet ett service account med behörighet att skriva till Firestore, vilket av säkerhetsskäl inte kan inkluderas i det publika Git-repot. Av denna anledning behöver en reproducerare av studien skapa ett eget firebase-projekt, generera en egen serviceAccountKey.json och köra seeding-scriptet för att återskapa en funktionellt identisk testmiljö. Instruktioner för detta finns dokumenterade i repots README på branch `parallel-db`.

2.3 Statistisk analys

Den statistiska analysen utgick från att varje mätning av den sekventiella implementationen matchades mot motsvarande mätning av den parallella implementationen för samma sida, iteration och databasstorlek. Analysen genomfördes därför som en parad jämförelse, där skillnaden i laddtid definierades som sekventiell minus parallell laddtid. Positiva differenser indikerar därmed kortare laddtid för parallella anrop.

Inför analysen exkluderades cold-boot-mätningar från den inferentiella jämförelsen, eftersom dessa i hög grad påverkas av initialisering och cache-relaterade uppstartseffekter och därmed inte speglar den stabila laddningsprestandan vid återkommande sidvisningar. Den primära analysen baserades i stället på warm-mätningar. Saknade värden hanterades med pairwise deletion, vilket innebär att endast kompletta par inkluderades i hypotesprövningen. Normalisering användes inte, eftersom den parade designen redan hanterar variationer mellan mätningar.

Som beskrivande statistik rapporterades antal mätningar, medelvärde, standardavvikelse, 95% konfidensintervall, nätverkslatens och p95-värden för respektive sida och implementation. För hypotesprövning användes ett parat Wilcoxon signed-rank-test, valt eftersom mätningarna är beroende och laddtider kan vara snedfördelade samt innehålla avvikande mätningar. Testet genomfördes separat för *Profile page* och *Leaderboard page* för att undvika att sidspecifika beteenden sammanblandas i ett gemensamt test. Signifikansnivån sattes till $\alpha = 0.05$ och den statistiska prövningen kompletterades med en bedömning av praktisk signifikans baserad på observerade skillnader i millisekunder.

3 Resultat

I detta avsnitt presenteras resultaten från mätningarna av både sekventiella och parallella anrop men även resultaten från den statistiska analysen av skillnaderna mellan de två implementationsstrategierna. Resultaten är uppdelade efter databasstorlek och sida, och inkluderar både beskrivande statistik och inferentiella testresultat.

3.1 Mätningar av sekventiella anrop

Utifrån mätdatan för de sekventiella anropen, innan implementeringen av förbättringen, samanställdes tabellen nedan som visar beskrivande statistik för varje databasstorlek och sida. Tabellen inkluderar medelvärde, standardavvikelse, 95% konfidensintervall, nätverkslatens och p95-värden för både profil- och leaderboard-sidorna.

Tabell 1: Beskrivande statistik för sekventiella anrop, uppdelat på databasstorlek och sida

Databasstorlek	Sida	n	Medelvärde (ms)	Standardavvikelse (ms)	95% KI (ms)	Nätverkslatens medel ± std	p95 (ms)
100	Profile page	150	699.98	66.61	689.23–710.73	22.00 ± 3.16	807.10
100	Leaderboard page	150	570.88	33.69	565.44–576.32	22.00 ± 3.16	612.00
500	Profile page	150	703.55	65.67	692.96–714.15	22.80 ± 3.27	807.55
500	Leaderboard page	150	563.13	48.28	555.34–570.92	22.80 ± 3.27	626.55
1000	Profile page	150	683.95	54.47	675.16–692.74	23.80 ± 3.63	797.65
1000	Leaderboard page	150	567.45	39.21	561.12–573.77	23.80 ± 3.63	613.20

3.2 Mätningar av parallella anrop

På samma sätt som för de sekventiella anropen samanställdes tabellen nedan för resultaten för de parallella anropen, efter implementering av förbättringen.

Tabell 2: Beskrivande statistik för parallella anrop, uppdelat på databasstorlek och sida

Databasstorlek	Sida	n	Medelvärde (ms)	Standardavvikelse (ms)	95% KI (ms)	Nätverkslatens medel ± std	p95 (ms)
100	Profile page	150	480.63	56.67	471.49–489.78	22.00 ± 3.16	580.10
100	Leaderboard page	150	459.33	35.98	453.52–465.13	22.00 ± 3.16	521.90
500	Profile page	150	495.91	68.91	484.79–507.03	22.80 ± 3.27	596.55
500	Leaderboard page	150	460.19	52.03	451.79–468.58	22.80 ± 3.27	584.40
1000	Profile page	150	473.54	60.17	463.83–483.25	23.80 ± 3.63	583.10
1000	Leaderboard page	150	459.88	43.05	452.93–466.83	23.80 ± 3.63	545.15

3.3 Jämförande analys

Förbättringsresultat:
Med värdena från Tabell 1 och Tabell 2 kunde förbättringen, absolut och relativ, beräknas för varje databasstorlek och sida. Effektstorleken kunde också beräknas, vilket gjordes med rangbiseriell korrelation (r_{rb}), och de specifika resultaten för varje databasstorlek och sida redovisas nedan:

Tabell 3: Jämförelse före och efter

Databasstorlek	Sida	Absolut förbättring (ms)	Relativ förbättring	r_{rb}
100	Profile page	219.35	31.34%	1.000
100	Leaderboard page	111.55	19.54%	0.995
500	Profile page	207.64	29.51%	1.000
500	Leaderboard page	102.95	18.28%	0.960
1000	Profile page	210.41	30.76%	1.000
1000	Leaderboard page	107.57	18.96%	0.965

3.4 Statistisk signifikans

Med den statistiska analysen med Wilcoxon signed-rank test kunde p-värden bestämmas för de olika databasstorlekarna och sidorna.

Tabell 4: Statistisk signifikans av förbättring för varje databasstorlek och sida

Databasstorlek	Sida	W-statistik	p-värde
100	Profile page	11325.00	1.14672×10^{-26}
100	Leaderboard page	11294.00	2.12553×10^{-26}
500	Profile page	11325.00	1.1469×10^{-26}
500	Leaderboard page	10949.50	1.46696×10^{-24}
1000	Profile page	11325.00	1.14655×10^{-26}
1000	Leaderboard page	11125.50	5.89925×10^{-25}

Det resultat som presenteras i Tabell 4 visar att alla förbättringar är statistiskt signifikanta, med p-värden under den valda tröskeln på 0.05.

3.5 Implementeringsanalys

Under implementeringen av parallella nätverksanrop gjordes följande observationer:

- Implementeringstid: uppskattningsvis under 1 timme (ingen exakt tidlogg fördes)
- Kodkomplexitet: Profil-sidan: +2 rader kod, Leaderboard-sidan: +4 rader kod
- Bieffekter: Inga tydliga bieffekter observerades under testning
- Kompatibilitet: Testad i Flutter + Firebase-miljö utan funktionsbortfall.

4 Diskussion

I detta avsnitt diskuteras resultatet av förbättringen i laddningstider efter implementering av parallella nätverksanropen i Neatify, med fokus på dess effektivitet, påverkan av databasstorlek och nätverkslatens, samt dess lämplighet i kontexten av en Flutter- och Firebase-baserad applikation. Vidare diskuteras metodens lämplighet, begränsningar och de praktiska implikationerna av resultaten.

4.1 Utvärdering av förbättringens effektivitet

De uppmätta förbättringarna i Tabell 3 visar att den absoluta förbättringen i laddningstid var mellan 102.95 ms och 219.35 ms, vilket motsvarar en relativ förbättring på mellan 18.28% och 31.34% beroende på databasstorlek och sida. Effektstorlekarna i den tabellen indikerar även att förbättringen var mycket stark, med r_{rb}-värden i intervallet 0.960 till 1.000, vilket tyder på att skillnaden i laddningstid mellan sekventiella och parallella anrop var både stor och konsekvent i den riktning som gynnar parallellisering. Dessa förbättringar är inte bara statistiskt signifikanta, som visas i Tabell 4, utan också praktiskt betydelsefulla, särskilt i kontexten av mobilapplikationer där användare är känsliga för laddningstider. Även om laddtiderna höll sig under den kritiska gränsen på 1 sekund [1], kan en förbättring på över 100 ms fortfarande ha en positiv inverkan på användarupplevelsen, särskilt när det gäller att minska känslan av latens och förbättra den övergripande responsiviteten i applikationen.

4.2 Effekt av databasstorlek och nätverkslatens

Förbättringen var konsekvent över samtliga testade databasstorlekar, vilket tyder på att parallellisering av nätverksanrop är en robust metod för att förbättra laddningstider vid olika belastningsnivåer. En svag tendens till större absolut förbättring observerades för den minsta databasen, men utifrån den insamlade datan kan detta inte tolkas som en tydlig trend.

Nätverkslatensen, som var relativt stabil över mätningarna, verkar inte ha haft någon större inverkan på förbättringens storlek. Dock kan inte någon slutsats dras om interaktionen mellan nätverkslatens och förbättringens effektivitet baserat på den data som samlats in, eftersom nätverkslatensen inte varierade tillräckligt mycket för att möjliggöra en sådan analys.

4.3 Lämplighet av parallella nätverksanrop för en Flutter- och Firebase-baserad applikation

Parallellisering av nätverksanrop gav tydliga prestandaförbättringar i form av kortare laddningstider och därmed en bättre användarupplevelse i Neatify. Metoden krävde endast begränsade kodändringar och påverkade inte applikationens funktionalitet eller stabilitet. Samtidigt förutsätter metoden att nätverksanropen är oberoende, och den kan göra felhantering något mer komplex, vilket är en viktig aspekt att överväga i mer komplexa applikationer än Neatify.

4.4 Metodkritik och begränsningar

Metoden bedöms ha varit lämplig för att besvara frågeställningen, eftersom den visade en statistiskt signifikant förbättring i laddningstider efter implementeringen av parallella nätverksanrop i den testade applikationen. Samtidigt finns det begränsningar som bör beaktas vid tolkningen av resultaten. Mätningarna genomfördes i en kontrollerad miljö med en specifik enhet, stabil Wi-Fi-anslutning och inaktiverad cachelagring. Detta ger en tydlig bild av förbättringens effekt under kontrollerade förhållanden, men innebär också att resultaten inte fullt ut speglar den variation som kan förekomma i verkliga användningsförhållanden, där enheter, nätverksförhållanden och användarbeteenden kan variera kraftigt.

Eftersom resultaten är knutna till Neatify och den aktuella implementationen av parallella nätverksanrop bör slutsatser om generaliserbarhet dras med försiktighet. Andra applikationer, särskilt de med mer komplexa beroenden mellan nätverksanrop eller som körs på mer resursbegränsade enheter, kan ge andra resultat.

4.5 Praktiska implikationer

Resultaten visar att parallellisering av oberoende Firestore-anrop kan ge tydliga och praktiskt relevanta förbättringar i laddningstid för Flutter- och Firebase-baserade applikationer. För Neatify innebär detta att sidan kan renderas snabbare utan att applikationens funktionalitet behöver förändras i grunden. Metoden är därför särskilt lämplig i situationer där flera databasanrop kan utföras oberoende av varandra och där snabbare sidladdning är viktig för användarupplevelsen.

Resultaten ligger i linje med tidigare forskning om parallell exekvering och I/O-bundna operationer, där total väntetid kan minskas genom att oberoende arbete utförs samtidigt i stället för sekventiellt. Samtidigt visar studien att förbättringens storlek varierar mellan olika sidor och databasstorlekar, vilket innebär att parallellisering inte bör betraktas som en universallösning, utan som en metod som bör användas när beroendena mellan anropen tillåter det.

Rekommendationer för implementation: Baserat på studiens resultat rekommenderas parallellisering av oberoende nätverksanrop i Flutter- och Firebase-baserade applikationer, särskilt när kortare laddningstid prioriteras och när anropen inte är beroende av varandras resultat.

Förslag på framtida forskning:

- Utvärdera metoden på fler typer av Flutter-applikationer med annan datastruktur och andra beroenden mellan anrop.
- Undersöka hur resultaten påverkas av varierande nätverksförhållanden och olika hårdvarukonfigurationer.
- Jämföra parallellisering med andra optimeringsmetoder, till exempel cachning eller reducering av antal anrop.
- Genomföra användarstudier för att undersöka hur de uppmätta förbättringarna upplevs i praktiken.

4.6 Slutsats

Studien visar att parallellisering av oberoende nätverksanrop med Future.wait() är en effektiv metod för att förbättra laddningstider i den testade Flutter- och Firebase-applikationen Neatify. Jämfört med sekventiell exekvering gav metoden genomgående kortare laddningstider för både Profile page och Leaderboard page, med absoluta förbättringar på cirka 103-219 ms och relativa förbättringar på cirka 18-31 procent beroende på sida och databasstorlek.

De observerade skillnaderna var statistiskt signifikanta i samtliga jämförelser enligt Wilcoxon signed-rank-test. Resultaten var dessutom stabila över de testade databasstorlekarna, utan någon tydlig trend som visar att databasstorlek i sig förändrade förbättringens storlek i denna mätserie.

Samtidigt bör resultaten tolkas i ljuset av studiens begränsningar: mätningarna genomfördes i en kontrollerad miljö med en testenhet, stabil Wi-Fi och inaktiverad cachelagring. Slutsatsen är därför att metoden är praktiskt relevant och lågtröskelig att införa i liknande applikationer, men att generalisering till andra miljöer och applikationstyper bör göras med försiktighet.

Referenser

[1] Jakob Nielsen. Response times: The 3 important limits. https://www.nngroup.com/articles/response-times-3-important-limits/, 1993. Accessed: 2026-04-04.

[2] Remzi H Arpaci-Dusseau and Andrea C Arpaci-Dusseau. *Operating systems: Three easy pieces*, volume 1. Arpaci-Dusseau Books, LLC Madison, WI, USA, 2018.

[3] Google. Flutter faq. https://docs.flutter.dev/resources/faq#:~:text=Flutter%20is%20Google%27s%20portable%20UI,is%20free%20and%20open%20source., 2026. Accessed: 2026-04-01.

[4] Dart. Future.wait. https://api.dart.dev/dart-async/Future/wait.html, 2026. Accessed: 2026-04-04.

[5] Dart. Asynchronous programming: futures, async, await. https://dart.dev/libraries/async/async-await, 2026. Accessed: 2026-04-04.

[6] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. Pearson, 8 edition, 2021.

[7] Google. Firebase documentation. https://firebase.google.com/docs, 2026. Accessed: 2026-04-01.

[8] Google. Cloud firestore documentation. https://firebase.google.com/docs/firestore, 2026. Accessed: 2026-04-02.

[9] Google. Firebase authentication documentation. https://firebase.google.com/docs/auth, 2026. Accessed: 2026-04-02.

[10] David DeWitt and Jim Gray. Parallel database systems: The future of high performance database systems. *Communications of the ACM*, 35(6):85–98, 1992.

[11] Yoon-Min Nam, Donghyoung Han, and Min-Soo Kim. A parallel query processing system based on graph-based database partitioning. *Information Sciences*, 480:237–260, 2019.

[12] Hao Li, Yi-Cheng Tu, and Bo Zeng. Concurrent query processing in a gpu-based database system. *PloS one*, 14(4):e0214720, 2019.

[13] Zhongqi Zhu. Research on parallel execution techniques for improving the expandability of database systems. *Journal of Computer, Signal, and System Research*, 2(3):69–74, 2025.